

HACKMTY 2026

The Forensic Auditor

DOCUMENTO DE ARQUITECTURA

versión 2 — sobre el schema oficial de los organizadores

Este documento es el contrato. Se acuerda en la primera hora.

0 — El stack

Backend	Python 3
Estate	SQLite (archivo .db)
Modelo	Ollama local + Gemini de respaldo
Frontend	Vite + React
Paralelo	Tiger Data y Vultr, solo categoría

IMPLICACIÓN DE DISEÑO — por qué SQLite y no Postgres

El validador oficial hace `sqlite3.connect` sobre el archivo del estate. El estate es un `.db` y punto.

Ventaja secundaria: cero instalación, funciona igual en las dos Mac y las dos Windows, y se puede copiar entre máquinas como cualquier archivo.

Tiger Data queda como despliegue paralelo: se carga una copia del estate allá para cumplir la categoría, sin que el pipeline dependa de ello.

IMPLICACIÓN DE DISEÑO — el presupuesto de 50 ms NO es nuestro

Esa regla es del track Courier. En Forensic lo que se reporta es `wall_clock_seconds`, sin límite fijo.

Aun así el demo son tres minutos, así que el objetivo práctico sigue siendo una investigación completa en menos de noventa segundos.

1 — Vista general

```
estate.db  (ruta dada en tiempo de ejecución)
|
DETECTORES      SQL, uno por tipo de esquema
|  candidatos ordenados
INVESTIGADOR     modelo + herramientas
|  hallazgo en borrador
RETADOR          modelo: intenta tumbarlo
|  sobrevive / se cierra
VALIDADOR        código: cita, reconcilia, cuenta
|
submission.json + case_file.html
```

Tres capas de filtro antes de que algo se imprima como acusación. Cada una puede cerrar un lead, y el que la cerró se registra en el campo `closed_by`.

Qué hace cada rol

Rol	Qué es	Qué produce
Investigador	Modelo. Elige a quién investigar, formula la hipótesis, llama herramientas.	Hallazgo en borrador con sus exhibits, o lead cerrado.
Retador	Modelo. Recibe el borrador y la evidencia. Su única tarea es construir la explicación inocente más fuerte posible.	Sobrevive, o se cierra con el argumento que lo tumbó.
Validador	Código. Sin modelo.	Verifica que cada <code>record_id</code> exista, que el monto reconcilie al 2%, que haya al menos 3 exhibits y que el money trail conecte.

IMPLICACIÓN DE DISEÑO — el validador es código, no modelo

Las reglas dicen que una acusación debe validar antes de imprimirse: cada `record_id` citado tiene que existir en el `estate` y el `peso_amount` tiene que reconciliar contra los exhibits dentro del 2%.

Eso no se le pide a un modelo. Se verifica con consultas. Si el validador falla, el hallazgo no se publica y pasa a `leads_not_pursued` con `closed_by: validator`.

IMPLICACIÓN DE DISEÑO — qué hace y qué no hace el modelo

SÍ: decide a quién investigar y en qué orden
formula la hipótesis
construye el argumento contrario (retador)
redacta el narrative y los reason

NO: calcular montos o porcentajes
decidir si un hallazgo se publica
tocar el ground truth

2 — El estate

Ocho tablas, definidas por los organizadores en `estate_schema.sql`. Los nombres de columna se copian tal cual: los jueces los leen directo al inspeccionar el estate, y cada exhibit cita un `source_table` por nombre.

vendors	rfc, legal_name, registered_date, address, bank_clabe, category, contact_email
invoices	uuid, issuer_rfc, receiver_rfc, issue_date, subtotal, iva, total, concepto_text, uso_cfdi, forma_pago, metodo_pago, status
ledger	entry_id, date, account_code, account_name, debit, credit, description, invoice_uuid, cost_center, approver
bank_txns	txn_id, date, from_clabe, to_clabe, amount, reference, channel
purchase_orders	po_id, vendor_rfc, date, amount, requester, approver, description
contracts	contract_id, vendor_rfc, start_date, value, scope_text
employees	emp_id, name, role, bank_clabe, hire_date
efos_list	rfc, legal_name, status, publication_date

Diferencias con nuestro diseño anterior

- **No hay tabla de cuentas.** El CLABE vive dentro de vendors y de employees. bank_txns usa from_clabe y to_clabe directo. El adaptador cuenta-a-RFC se simplifica a una búsqueda por CLABE.
- **No hay tabla de conceptos.** concepto_text es un campo de texto libre en invoices.
- **No hay evidencia de entrega.** Solo contracts y purchase_orders. Nuestro gate de materialidad pierde el hueco de en medio: queda contrato, orden de compra con su aprobador, y transferencia.
- **efos_list solo tiene definitivo y presunto.** Los cuatro estatus no existen en este schema. El manejo de casos ambiguos pasa a depender de los decoys.

IMPLICACIÓN DE DISEÑO — el CLABE tiene estructura y ahí vive un decoy

Un CLABE mexicano son 18 dígitos: los primeros tres son el código del banco, los siguientes tres la plaza, once de cuenta y uno verificador.

El README de los organizadores pregunta literalmente: «¿y si el empleado simplemente tiene su cuenta en el mismo banco?». Eso es una pista.

Consecuencia: el vínculo empleado-proveedor se comprueba con el CLABE completo, nunca por prefijo. Un detector que empate por los primeros dígitos va a acusar a todo el que use el mismo banco.

Y al revés: sembramos ese decoy a propósito. Un empleado y un proveedor en la misma institución, con cuentas distintas, es exactamente la entidad honesta que dispara un detector ingenuo.

El ground truth

No es una tabla del estate. Es un archivo JSON aparte, con el formato de `ground_truth_schema.json`: `seed`, `company_rfc`, `schemes` y `decoys`.

```
out/
  estate_seed001.db      <- lo que ve el agente
  truth_seed001.json    <- lo que NO ve
```

IMPLICACIÓN DE DISEÑO — aislamiento verificable

Los jueces pueden correr: `grep -r 'ground_truth' src/ --include='*.py'`

Si el ground truth aparece en el agente, en sus herramientas, o en cualquier cosa que importen, Results topa en 2 sin importar los números.

Regla del equipo: la palabra `ground_truth` solo puede aparecer en `generate/` y en `eval/`. Nunca en `tools/` ni en `agent/`.

Conviene poner ese `grep` en el build, junto al validador oficial.

3 — El generador

Toma un seed y produce un estate .db más su ground truth. Es la pieza más grande del proyecto y toda la medición depende de ella.

```
python -m generate --seed 1 --out out/  
  
-> out/estate_seed001.db  
-> out/truth_seed001.json
```

Todo aleatorio pasa por una única instancia sembrada. Nada de random global, nada de fecha del sistema, nada de uuid4 sin semilla. El mismo seed tiene que reproducir el mismo archivo.

Los cinco esquemas

Tipo	Cómo se siembra	Cómo se detecta
phantom_vendor	Proveedor sin contrato ni orden de compra, dado de alta poco antes de su primera factura, concepto genérico, y en efos_list.	Cruce con efos_list más ausencia de contracts y purchase_orders.
kickback	El proveedor recibe el pago y transfiere parte al CLABE de un empleado.	bank_txns cuyo to_clabe aparece en employees.
round_tripping	El dinero sale de la empresa, pasa por dos o tres proveedores y regresa al CLABE de la empresa.	Recorrido recursivo sobre bank_txns que cierra el ciclo.
threshold_splitting	Varias facturas del mismo proveedor, fechas cercanas, cada una justo debajo del límite de aprobación.	Agrupar por proveedor y ventana de fechas; comparar contra el límite inferido de purchase_orders.
revenue_inflation	La empresa emite facturas por ventas que nunca cobró.	invoices donde issuer_rfc es la empresa y no existe bank_txn que las liquide.

Los decoys

Entidades honestas que disparan un detector y se limpian al inspeccionarlas. Acusar a una cuenta como falsa acusación, y eso pesa al menos tanto como el recall.

1. Proveedor en efos_list con estatus presunto, pero con contrato vigente, orden de compra y pago único sin retorno.
2. Empleado y proveedor en el mismo banco: mismos tres dígitos iniciales de CLABE, cuentas distintas. Sin transferencia entre ellos.
3. Facturación recurrente bajo el límite de aprobación por una razón legítima: contrato marco con cuota mensual fija.
4. Proveedor que también es cliente: hay dinero de vuelta, pero con facturas propias en

sentido contrario.

5. Proveedor dado de alta poco antes de su primera factura, pero con contrato firmado antes del alta.

Los jueces pueden incluir hasta diez decoys en un estate, y hasta los cinco tipos de esquema. También esquemas entrelazados: dos que comparten una entidad, de modo que un money trail cruza la frontera entre ellos.

IMPLICACIÓN DE DISEÑO — cada decoy existe para tumbar un detector específico

No se siembran decoys genéricos. Cada uno se diseña contra uno de nuestros propios detectores, y la razón por la que se limpia es exactamente el texto que el agente debería escribir en `leads_not_pursued`.

Eso hace que la sección de leads descartados tenga contenido específico en vez de «evidencia insuficiente», que las reglas califican como genérico.

IMPLICACIÓN DE DISEÑO — un seed sin fraude

Al menos un seed del conjunto de reserva no lleva ningún esquema sembrado, solo decoys.

Una lista de hallazgos vacía es un resultado legítimo según el formato oficial, y un sistema que siempre encuentra algo es un sistema que siempre acusa.

4 — Las herramientas

Ocho funciones sobre SQLite. Devuelven datos, nunca juicios.

```
prioritized_candidates() -> list
    detectores deterministas, sin modelo.
    orden estable: por señal y desempate por rfc.

vendor(rfc) -> dict
    fila de vendors más su estatus en efos_list
    con publication_date.

invoices_by(rfc, role) -> list
    role = issuer | receiver.
    el segundo es lo que necesita
    revenue_inflation.

support_for(rfc) -> dict
    contracts y purchase_orders del proveedor,
    con montos y aprobadores.

trace_money(clabe, max_hops=4) -> dict
    WITH RECURSIVE sobre bank_txns.
    devuelve la ruta, el monto que vuelve,
    y el CLABE destino.

whose_clabe(clabe) -> dict
    a quién pertenece: vendors, employees
    o la propia empresa. CLABE completo,
    nunca por prefijo.

reciprocity(rfc_a, rfc_b) -> dict
    ¿hay facturas en sentido contrario que
    justifiquen el retorno? folios y montos.

ledger_for(uuid) -> list
    asientos contables de esa factura,
    con cost_center y approver.
```

IMPLICACIÓN DE DISEÑO — whose_clabe es la herramienta que evita el falso positivo

Es la que distingue «el proveedor le transfirió a un empleado» de «el proveedor y el empleado usan el mismo banco».

Si se implementa por prefijo, el sistema acusa a todo el que banquee en la misma institución. Los organizadores preguntan justo eso en su lista de preguntas del jurado.

IMPLICACIÓN DE DISEÑO — reciprocity es la que salva al proveedor-cliente

Un retorno de dinero puede ser legítimo: compraventa recíproca, devoluciones, reembolsos. Lo que lo distingue es que el retorno legítimo tiene su propia factura.

Sin esta herramienta, el agente acusa a todo proveedor que también sea cliente.

5 — El ciclo

1. `prioritized_candidates()`
2. por cada candidato, el INVESTIGADOR:
 - consulta herramientas
 - arma hallazgo en borrador:
 - `scheme_type`, `entities`, `exhibits`,
 - `peso_amount`, `money_trail`
 - o lo cierra -> `closed_by: investigator`
3. el RETADOR recibe borrador + evidencia:
 - "construye la explicación inocente más fuerte que encaje con estos datos"
 - si la explicación se sostiene:
 - cierra -> `closed_by: challenger`
 - si no:
 - el hallazgo sobrevive, y se guarda
 - qué argumentó y por qué no bastó
4. el VALIDADOR, en código:
 - cada `record_id` existe en el estate
 - `peso_amount` reconcilia al 2%
 - al menos 3 exhibits
 - el `money_trail` conecta: el destino de cada paso es el origen del siguiente
 - narrative bajo 150 palabras
 - `entities` con prefijo RFC: o EMP:
 - si algo falla -> `closed_by: validator`
5. serializar `submission.json` y `case file`

IMPLICACIÓN DE DISEÑO — lo que argumentó el retador va en el case file

La estructura oficial del case file pide que, si hay revisión adversarial, se incluya qué se argumentó y por qué el hallazgo sobrevivió, porque un hallazgo que nadie intentó romper es más débil que uno atacado que resistió.

Así que el argumento del retador no es un descarte interno: es contenido publicable y suma en Judgment.

Reconciliación de pesos

El validador oficial suma los montos de los exhibits citados por tabla, no entre tablas, y compara contra el `peso_amount` reclamado con tolerancia del 2%.

La razón es que una factura y la transferencia que la liquidó son los mismos pesos vistos dos veces. Citar el `money trail` completo no penaliza.

Tablas que llevan monto: `invoices` con `total`, `bank_txns` con `amount`, `purchase_orders` con `amount`, `contracts` con `value`. Cada hallazgo debe citar al menos una de ellas o el monto no puede reconciliar.

Determinismo

Generador	una sola instancia de random sembrada con el seed
Modelo	temperatura 0
Caché	clave = sha256 del prompt, persistida en disco
Candidatos	orden estable, desempate por rfc
Validador	código puro

El caché es también el mecanismo de replay sin red: en modo grabación escribe, en modo replay solo lee. Los jueces pueden pedir reproducir una corrida con la conexión apagada.

Los tres números

Se cuentan en el envoltorio del cliente del modelo, no a mano.

llm_calls	contador de llamadas
mxn_cost	0.0 con Ollama local
wall_clock_seconds	cronómetro de la corrida

Que el costo sea cero es un argumento de Feasibility, no una carencia: un despacho puede correr esto en su propia red, sin que los datos del cliente salgan a ninguna API.

6 — La salida

Dos artefactos. El JSON se revisa por máquina; el case file lo lee un juez.

El case file, cinco secciones

1. Encabezado
empresa, periodo, seed, y los tres números.
Si la corrida es determinista, se dice.
2. Resumen ejecutivo
qué se encontró, en lenguaje llano, con
tabla de hallazgos, exposición total y
leads cerrados.
3. Una sección por hallazgo
encabezado, regla rota, monto y confianza,
qué pasó en menos de 150 palabras,
MONEY TRAIL COMO DIAGRAMA,
tabla de exhibits, y la aritmética de
la reconciliación.
Más lo que argumentó el retador.
4. Leads no perseguidos
en el cuerpo, no en anexo. Entidad, qué
detector la señaló, la razón específica
citando la evidencia, qué herramientas se
llamaron, y quién lo cerró.
5. Método y límites
arquitectura en unas frases, qué quedó
fuera de alcance, qué el sistema NO puede
detectar, y cómo reproducir el archivo.

IMPLICACIÓN DE DISEÑO — el money trail en prosa topa la Clarity en 3

Tiene que renderizar como diagrama, y cada paso citar un exhibit_id que exista en la lista de exhibits del hallazgo.

Y tiene que producirse sin conexión. SVG generado localmente, embebido en el HTML. Nada de librerías por CDN ni servicios de render en línea.

Estructura del proyecto

src/	
generate/	generador del estate y del ground truth. Único lugar, junto con eval/, donde puede aparecer la palabra ground_truth.
tools/	las 8 herramientas SQL
agent/	investigador, retador, validador

report/	submission.json y case file
cli.py	recibe la ruta del estate
eval/	harness de métrica
out/	estates y ground truths
frontend/	Vite + React

La ruta del estate se recibe como argumento. Rutas hardcodeadas fallan según las reglas.

En el build, desde el inicio

```
python3 validate_format.py --submission out/sub.json \  
                           --estate out/estate_seed001.db  
  
grep -r 'ground_truth' src/ --include='*.py'  
# solo debe aparecer en src/generate/
```

7 — Lo primero

Cuándo	Qué	Quién
Ahora	Correr <code>validate_format.py</code> contra <code>submission_example.json</code> . Confirmar que pasa. Ese es el contrato de salida.	Todos
Primera hora	Copiar <code>estate_schema.sql</code> tal cual al proyecto. No renombrar nada.	Estate
Primera hora	Acordar los seeds: tuning 1-10, reporte 101-110. Los de reporte no se tocan.	Todos
Primera hora	Envoltorio del cliente del modelo con contador, caché por hash y temperatura 0. Todo lo demás lo usa.	Agente
Desde la 1 h	Agente y salida trabajan con un estate mínimo escrito a mano. No esperan al generador.	Agente, Salida

Recordatorio

Ningún hallazgo se imprime sin haber sido atacado.

Investigador arma. Retador tumba. Validador verifica.
Lo que sobrevive, se publica con su prueba.